Programação funcional - lista 1

1. O que caracteriza o paradigma funcional e quais suas vantagens?

2. O que é uma função pura?

3. O que é recursão na cauda?

4. O que é cálculo λ?

5. Qual é o resultado da seguinte expressão λ? (\x -> x) y
   R: y

6. Implemente uma função que receba 2 listas ordenadas e realiza a fusão delas, resultando em uma única lista ordenada.

```
merge :: Ord a => [a] -> [a] -> [a]
merge xs [] = xs
merge [] ys = ys
merge (x:xs) (y:ys)
  | x <= y = x : (merge xs (y:ys))
  | otherwise = y : (merge (x:xs) ys)
```

7. Implemente uma função que calcule o produto cartesiano de 2 listas.

```
prodInt :: Num a => [a] -> [a] -> a
prodInt xs ys = sum [x*y | (x,y) <- zip xs ys]
```

8. O que a seguinte função (que) faz? Obs: a função **words** recebe uma string como parâmetro e retorna uma lista de strings composta pelas substrings da string original separadas por espaços e tabs.
   R.: Conta o número de ocorrências da palavra **p** no parágrafo **t**.

```
que :: String -> String -> Int
que p t = length
  $ filter (==p)
  $ words
  $ filter (/='.')
  $ filter (/='!')
  $ filter (/='?') t
```

9. Crie um tipo de dados utilizando o Record type para produtos de um mercado com os campos: código, nome, peso e preço. Também crie uma função que retorne o código do produto com melhor custo/benefício em relação ao preço e peso.

```
data Produto = MkProduto {
  cod :: Int,
  nome :: String,
  peso :: Double,
  preco :: Double
}

maisBarato :: Produto -> Produto -> Int
maisBarato p1 p2
  | (preco p1 / peso p1) <= (preco p2 / peso p2) = cod p1
  | otherwise = cod p2
```